

Warehouse Bots: Sequential, Parallel and Distributed Implementations

Luka Risteski

University of Primorska, FAMNIT
Koper, Slovenia

Abstract

This paper presents three implementations of a warehouse robot simulation: a sequential implementation, a shared-memory parallel implementation using Java threads, and a distributed implementation using MPJ Express.

The simulation contains autonomous warehouse robots that execute generated tasks, move through a shared warehouse environment and interact with shelves and other robots. The same instruction files are reused across all implementations, allowing their correctness and performance to be compared directly.

The implementations are evaluated using small, medium and large workloads. The parallel implementation is tested with one, two, four and eight threads, while the distributed implementation is tested with one, two, four and eight MPI processes.

The results show that the sequential implementation performs best for small workloads because parallelism introduces overhead that outweighs its benefits. Medium workloads benefit most from two workers, while the largest workload performs best with four workers. Increasing the worker count to eight generally reduces performance because of synchronisation, scheduling and communication overhead.

1 Introduction

Modern warehouses increasingly use autonomous robots to transport products, move shelves and complete delivery tasks. Simulating such systems is useful for studying robot coordination, collision avoidance, task scheduling and system scalability.

The purpose of this project is to implement the same warehouse simulation using three execution models:

- sequential execution,
- shared-memory parallel execution,
- distributed execution using message passing.

The sequential version executes all robot actions using a single thread. The parallel version distributes robot computations among multiple worker threads using Java's `ExecutorService`. The distributed version uses MPJ Express to divide work among multiple processes.

The main objective is to determine whether parallel and distributed execution improve performance and how performance changes as the number of threads or MPI processes increases.

The project also includes a graphical interface for observing the simulation and a headless execution mode for performance measurement.

2 System Description

The simulation models a warehouse as a two-dimensional grid containing robots, shelves, destinations and traversable cells.

Each robot receives a sequence of tasks from an instruction file. A typical task requires the robot to:

- (1) move to a specified shelf,
- (2) interact with the shelf,
- (3) transport an item to a destination,
- (4) continue with its next task.

The simulation advances in discrete ticks. During each tick, every active robot determines its next action.

Several robots may attempt to move simultaneously. Therefore, the simulation must prevent:

- two robots from occupying the same cell,
- robots from moving through obstacles,
- invalid shelf interactions,
- inconsistent updates to the shared state.

The simulation terminates when all robots have completed their assigned tasks.

The main components of the system are:

- GameState, which stores the complete simulation state,
- robot objects, which store robot positions and task queues,
- warehouse and shelf objects,
- an instruction parser,
- a task and instruction generator,
- graphical and headless execution modes.

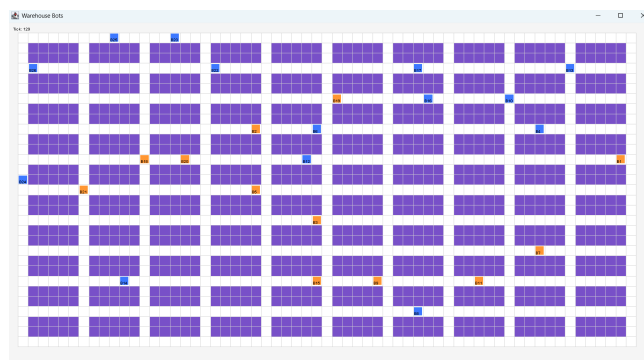


Figure 1: Graphical representation of the warehouse simulation during execution. Purple regions represent shelves, while labelled cells represent active robots.

2.1 Instruction Generation

The instruction generator creates reproducible workloads using fixed random seeds. A fixed seed causes the generator to produce the same warehouse layout and robot task sequence whenever it is executed with the same parameters.

The following seeds were used in the final evaluation:

2026, 98765, 555, 1001.

Seed 12345 was excluded from the performance averages because the largest workload did not terminate in the sequential and parallel implementations. This case is treated as a termination limitation rather than as a valid timing result.

3 Implementation

3.1 Sequential Implementation

The sequential implementation uses a single thread to process the entire simulation.

During each tick, the program iterates through the robots, determines their actions, validates the requested movements and updates the shared game state.

A simplified version of the sequential execution cycle is:

- (1) load the warehouse and robot instructions,
- (2) initialise the game state,
- (3) process every robot during the current tick,
- (4) validate and apply the calculated actions,
- (5) increment the simulation tick,
- (6) repeat until all robots finish.

Because no worker threads or messages are required, this implementation has very little coordination overhead. However, robot computations cannot be performed simultaneously.

3.2 Parallel Implementation

The parallel implementation uses shared-memory multithreading. Robot computations are submitted to a fixed-size thread pool implemented using Java's `ExecutorService`.

The number of worker threads can be configured before execution. The evaluation uses one, two, four and eight threads.

At each tick, the main thread submits work for the active robots. Worker threads calculate robot actions in parallel. The main thread waits for the required computations to finish before conflicts are resolved, the shared state is updated and the next tick begins.

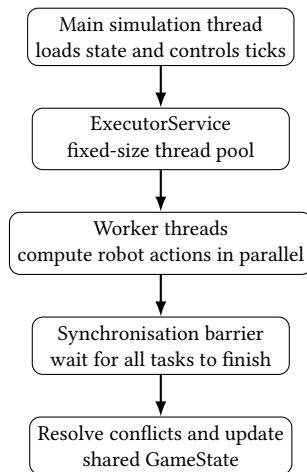


Figure 2: Architecture of the shared-memory parallel implementation.

All robots access the same warehouse state. Therefore, every tick requires synchronisation before the next tick can begin.

The parallel implementation introduces overhead from:

- task submission,
- thread scheduling,
- synchronisation,
- locking and shared-state contention,
- waiting at tick boundaries.

For small workloads, this overhead may be greater than the time saved through parallel execution.

3.3 Distributed Implementation

The distributed implementation uses MPJ Express, a Java implementation of the Message Passing Interface model.

The program is launched with a configurable number of MPI processes. Each process receives a unique rank. Rank 0 coordinates the simulation, while work is divided among the available worker processes.

A simplified distributed tick consists of the following steps:

- (1) rank 0 distributes the current state or required work,
- (2) worker processes calculate assigned robot actions,
- (3) calculated actions are returned to rank 0,
- (4) rank 0 resolves conflicts and updates the state,
- (5) the next simulation tick begins.

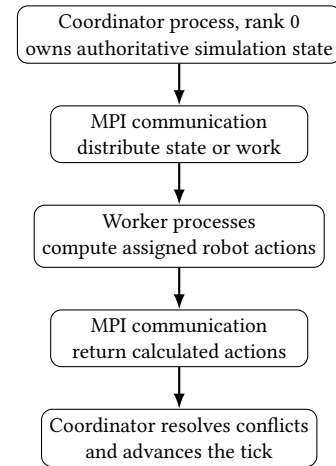


Figure 3: Architecture of the distributed MPJ Express implementation.

The distributed implementation introduces communication, serialisation and process-management overhead. Since all MPI processes were executed on one physical computer, they also compete for the same processor and memory resources.

Consequently, increasing the number of processes does not always improve performance.

4 Experimental Methodology

Three workload sizes were evaluated.

Table 1: Evaluated warehouse workloads.

Workload	Robots	Tasks	Warehouse
Small	10	5	5×5
Medium	25	10	8×8
Large	50	10	10×10

Each workload was generated using the four retained random seeds. Identical instruction files were reused when comparing different execution models and worker counts.

The parallel implementation was tested using $p \in \{1, 2, 4, 8\}$ threads, while the distributed implementation was tested using $p \in \{1, 2, 4, 8\}$ MPI processes. Each configuration was executed once for each of the four retained seeds. The reported result is the mean of those four executions.

All benchmarks were executed in headless mode to remove the overhead introduced by graphical rendering and animation. The graphical mode was used only during development to verify robot behaviour and the warehouse state visually.

Execution time was measured immediately before and after the simulation loop using a high-resolution timer.

Performance was evaluated using average execution time and speedup.

Speedup is defined as:

$$S_p = \frac{T_{\text{seq}}}{T_p}, \quad (1)$$

where T_{seq} is the average sequential execution time and T_p is the execution time obtained using p worker threads or MPI processes.

A speedup greater than 1 indicates that the implementation is faster than the sequential version. A speedup below 1 indicates that the coordination overhead is greater than the performance benefit.

The final simulation tick was also recorded as a correctness metric. Matching final ticks indicate that implementations required the same number of simulation steps for a given workload. Small differences may occur if simultaneous robot actions are resolved in a different order.

5 Results

The values in the following tables are means across one execution for each of the four retained random seeds.

5.1 Sequential Results

Table 2: Average sequential execution time.

Workload	Average time (ms)
Small	234.25
Medium	2500.25
Large	10928.25

The sequential implementation performs particularly well for the small workload because it avoids thread-management and communication overhead.

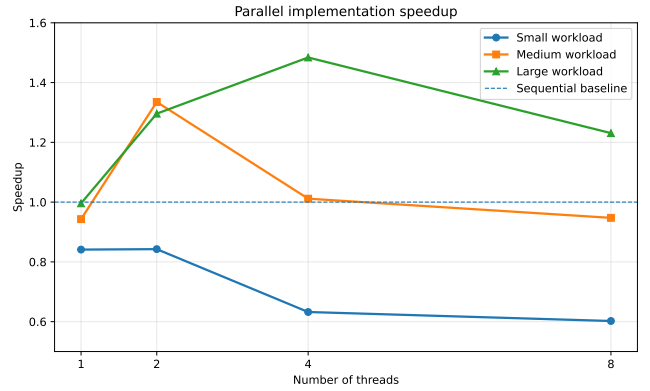
5.2 Parallel Results

Table 3: Average execution time of the parallel implementation in milliseconds.

Threads	Small	Medium	Large
1	278.50	2651.25	10972.50
2	278.00	1872.75	8432.25
4	370.50	2471.75	7364.00
8	389.00	2639.25	8878.25

For the small workload, no parallel configuration outperforms the sequential implementation.

For the medium workload, two threads provide the best average execution time. For the large workload, four threads provide the best result.

**Figure 4: Speedup of the parallel implementation relative to the sequential baseline.**

5.3 Distributed Results

Table 4: Average execution time of the distributed implementation in milliseconds.

MPI processes	Small	Medium	Large
1	240.50	2438.50	10613.50
2	293.25	1907.50	8563.25
4	445.00	2659.75	8051.50
8	661.75	3511.25	10431.50

One MPI process is best for the small workload. Two processes are best for the medium workload, while four processes are best for the large workload.

Eight processes perform poorly for all workload sizes because the simulation does not contain enough independent computation to compensate for communication, synchronisation and process-management overhead.

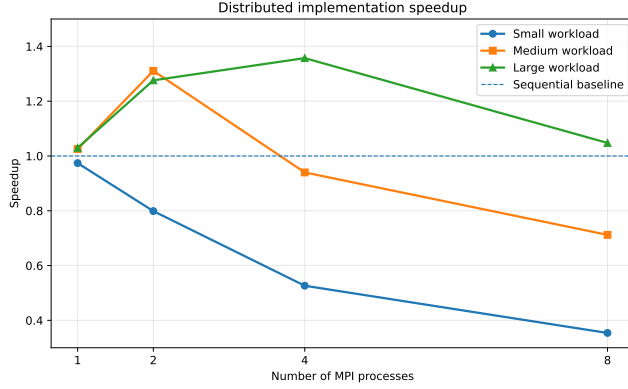


Figure 5: Speedup of the distributed implementation relative to the sequential baseline.

5.4 Best Configuration Comparison

The best-performing configuration for each implementation is shown in Figure 6. The sequential implementation performs best for the small workload, whereas the parallel implementation achieves the lowest execution times for both the medium and large workloads.

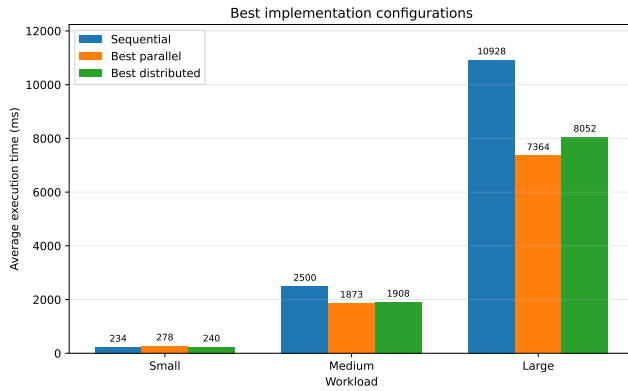


Figure 6: Comparison of the fastest measured configuration of each implementation.

6 Discussion

The benchmark results show that the effectiveness of parallelism depends strongly on the amount of work performed during each simulation tick.

Small workload. The sequential implementation performs best for the smallest workload. The amount of computation performed by the robots is too small to compensate for task submission, scheduling, synchronisation and MPI communication overhead.

As a result, all parallel configurations produce speedups below 1. The distributed implementation also becomes progressively slower as additional MPI processes are added.

Medium workload. As the workload increases, the computation performed by each worker becomes large enough to compensate for coordination overhead.

The best results are achieved using two worker threads and two MPI processes. The two-thread implementation reduces execution time from 2500.25 ms to 1872.75 ms.

Its speedup is:

$$S_2 = \frac{2500.25}{1872.75} \approx 1.34. \quad (2)$$

The two-process distributed implementation achieves a similar result, with an average execution time of 1907.50 ms.

Increasing the number of workers to four or eight reduces performance, showing that the medium workload does not contain enough parallel work to use the additional workers efficiently.

Large workload. The largest workload benefits most from parallel execution. The four-thread implementation reduces average execution time from 10928.25 ms to 7364.00 ms.

Its speedup is:

$$S_4 = \frac{10928.25}{7364.00} \approx 1.48. \quad (3)$$

The four-process distributed implementation also improves performance, but remains slower than the four-thread shared-memory version.

This is expected because shared-memory threads can access the same state directly, while MPI processes must exchange information through messages.

Overall observations. Increasing the worker count from four to eight generally reduces performance. Additional workers increase scheduling, synchronisation and communication costs while the amount of useful parallel work remains limited.

The program also contains sequential sections, including:

- conflict resolution,
- state updates,
- tick coordination,
- input parsing,
- simulation termination checks.

These sequential sections limit the maximum achievable speedup, which is consistent with Amdahl's law.

The sequential and parallel implementations generally produce matching final ticks. Small differences may occur in the distributed implementation because simultaneous actions can be evaluated and resolved in a different order.

All retained benchmark workloads nevertheless completed successfully and produced consistent overall behaviour.

7 Conclusion

This project presented sequential, shared-memory parallel and distributed implementations of a warehouse robot simulation.

The experimental results show that no single execution model is best for every workload size. The sequential implementation provides the best performance for small workloads because it introduces minimal coordination overhead.

The medium workload performs best with two worker threads, while the large workload performs best with four worker threads. The four-thread parallel implementation achieves the best overall result, with an average speedup of approximately 1.48.

The distributed implementation also improves performance for medium and large workloads, but it remains slower than the parallel version on the same physical computer because of communication and process-management overhead.

Using eight threads or MPI processes generally reduces performance. Therefore, increasing the number of workers does not automatically produce a faster program. The amount of useful parallel computation must be large enough to compensate for scheduling, synchronisation and communication costs.

Overall, the shared-memory implementation provides the strongest performance for the evaluated medium and large workloads, while the sequential implementation remains the best choice for small workloads.